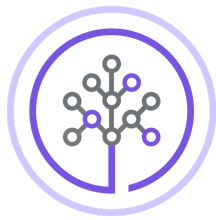


Lab: Build a Full MCP Application



Skills
Network

Estimated time: 30 minutes

Introduction

In this lab, you will build a complete MCP host application that integrates a Gradio chat interface with a WatsonX LLM. Using a ReAct agent loop, the system analyzes user questions, determines which MCP tools are needed, invokes them, and converts the results into clear natural language responses.

Screenshot requirement for the Final Project

During the course of the lab, you will be instructed to take a screenshot showing the full Gradio UI, including the application header, a visible chat history with at least one exchange, input/action buttons, and complete branding/footer. Name the file **M4L3_Design_LLM_MCP_Host.jpg**.

This screenshot will later serve as a component for the Final Project.

Learning objectives

By the end of this lab, you will have:

- Built an **MCP host** that discovers server tools at runtime and binds them to an LLM
- Implemented a **ReAct agent loop** that calls MCP tools iteratively until a final

- Wired the agent into a **Gradio chat UI** with quick-start prompt buttons
- Run the full application end-to-end from a single command

Setup

Install the required libraries:

 bash 

```
pip install gradio==6.9.0 fastmcp==3.1.0 langchain-ibm==1.0.4 langchain-core=
```

 Run

Download the data files and the server from the previous lab:

 bash 

```
curl -O https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/q5  
  
curl -O https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/ta  
  
curl -O https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/q1  
  
curl -O https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/cl
```

 Run

Create the application file:

 bash 

```
touch app.py
```

 Run

Open app.py in IDE

MCP Host foundation

Imports and configuration

At the top of `app.py`, import the required libraries and set up the configuration values the host needs.

Note: in this section you will need to write the system prompt for the agent running the application. In the system prompt, make sure to inform the agent of the following:

- Its name is `Connoisseur Companion`
- Has access to a database of california restuarants
- It can use `get_restaurant_info` to look up specific restaurants by name
- It can use `recommend_by_vibe` to find restaurants matching a mood or atmosphere
- It can use `get_review` to retrieve detailed reviews of restaurants

Locate the line that is marked as `TODO` and replace the `...` in the code with your own message.

Hint: you might benefit from using triple quotation marks `"""` as the system prompt will be multi-lined

python

```
# Libraries to create our MCP host application
import os
import gradio as gr
from pathlib import Path
from fastmcp.client import Client, PythonStdioTransport
from langchain_ibm import ChatWatsonx
from langchain_core.messages import HumanMessage, AIMessage, ToolMessage, Sys

# Configuration
SERVER_SCRIPT = str(Path(__file__).parent / "server.py")
SYSTEM_PROMPT = ... # TODO

project_id = (
    os.environ.get("WATSONX_AI_PROJECT_ID")
    or os.environ.get("WATSONX_PROJECT_ID")
)
```

`SERVER_SCRIPT` points to the MCP server we built in the previous lab—the host will launch it as a subprocess. `SYSTEM_PROMPT` defines the agent's persona and instructs it on when to use each tool. `project_id` is read from an environment variable, so API credentials are never hard-coded.

Initializing the WatsonX LLM

Next, define the factory function that creates the language model.

python

```
# Initializing the WatsonX LLM
def make_model():
    return ChatWatsonx(
        model_id="ibm/granite-4-h-small",
        url="https://us-south.ml.cloud.ibm.com",
        project_id=project_id,
        params={"temperature": 0.7},
    )
```

`make_model` is a factory rather than a module-level constant, so a fresh model instance is created for each conversation turn—this avoids subtle state issues between concurrent requests. `temperature=0.7` gives responses some variety while keeping them focused.

The ReAct agent loop

We are still working on `app.py`, so continue to add all the following code to it.

The heart of the application is the **ReAct (Reason + Act) loop**: the LLM reasons about what it needs, calls a tool, observes the result, and repeats—until it can produce a final text response with no further tool calls.

MCP Host: chat_with_agent



python



```
# MCP Host – ReAct Agent Loop
async def chat_with_agent(user_message: str, history: list) -> str:
    """Connect to the MCP server, discover tools, and run a ReAct loop.
    The LLM decides which tools to call, calls them via the MCP server,
    and repeats until it produces a final text response."""
    transport = PythonStdioTransport(script_path=SERVER_SCRIPT)

    async with Client(transport) as client:
        # Discover available tools from the MCP server
        mcp_tools = await client.list_tools()

        # Convert MCP tool schemas to OpenAI-style tool definitions for the L
        openai_tools = [
            {
                "type": "function",
                "function": {
                    "name": t.name,
                    "description": t.description or "",
                    "parameters": t.inputSchema,
                },
            },
            for t in mcp_tools
        ]

    model = make_model().bind_tools(openai_tools)

    # Build the message list from chat history and the new user message
    messages = [SystemMessage(content=SYSTEM_PROMPT)]
    for msg in history:
        role = msg.get("role", "")
        content = msg.get("content", "")
        if role == "user" and content:
            messages.append(HumanMessage(content=content))
        elif role == "assistant" and content:
            messages.append(AIMessage(content=content))
    messages.append(HumanMessage(content=user_message))

    # ReAct loop – call tools until the LLM returns a plain text reply
    for _ in range(10):
        response = await model.ainvoke(messages)
        messages.append(response)
```



```

raw = response.content
if isinstance(raw, list):
    return " ".join(
        b.get("text", "") if isinstance(b, dict) else str(b)
        for b in raw
    )
return str(raw)

# Execute each tool call via the MCP server and feed results back
for tool_call in response.tool_calls:
    result = await client.call_tool(tool_call["name"], tool_call[
    tool_output = " ".join(
        item.text if hasattr(item, "text") else str(item)
        for item in result.content
    ) if result.content else "(no result)"
    messages.append(ToolMessage(content=tool_output, tool_call_id

return "I wasn't able to complete that request. Please try again."

```

There are four key steps inside this function:

1. **Tool discovery:** `client.list_tools()` asks the MCP server which tools it exposes. This happens at runtime, so the host automatically picks up any tools added to the server without needing code changes.
2. **Schema conversion:** MCP tool schemas are converted to the OpenAI-style format that LangChain's `bind_tools` accepts. This bridges the MCP protocol and the LLM's tool-calling interface.
3. **Message reconstruction:** The existing chat history is converted from Gradio's dict format into LangChain message objects, so the LLM maintains conversation context across turns.
4. **ReAct iteration:** Each pass through the loop invokes the model. If the model returns `tool_calls`, each is executed against the MCP server and its output is appended as `ToolMessage` before the next iteration. When the model returns a plain text response with no tool calls, the loop exits, and the answer is returned.

Gradio interface

Event handler

The Gradio event handler wraps `chat_with_agent` and manages the optimistic UI update that shows a "Thinking..." placeholder while the agent runs.

```
# Gradio Event Handler
async def handle_chat(user_message, history):
    if history is None:
        history = []
    if not user_message or not user_message.strip():
        yield history
        return

    # Show a thinking placeholder while the agent runs
    history = history + [
        {"role": "user", "content": user_message},
        {"role": "assistant", "content": "Thinking..."},
    ]
    yield history

    response_text = await chat_with_agent(user_message, history[:-2])
    history[-1] = {"role": "assistant", "content": response_text}
    yield history
```

Using `yield` makes `handle_chat` a Python async generator. Gradio streams each yielded value to the browser, so the user immediately sees their message and the "Thinking..." placeholder appear, and then sees the real answer replace the placeholder once the agent finishes.

Building the UI

Finally, define the Gradio layout and wire up all the interactive elements.



python



```
# Gradio Interface
with gr.Blocks(title="Connoisseur Companion") as demo:
    gr.Markdown("# Connoisseur Companion\nYour AI guide to California's resta

    chatbot = gr.Chatbot(height=500)
    msg_input = gr.Textbox(
        label="Ask about restaurants",
        placeholder='e.g., "Find me a moody spot in DTLA" or "Tell me about S
    )

    with gr.Row():
        btn1 = gr.Button("Find moody restaurants", size="sm")
        btn2 = gr.Button("Tell me about Iron & Embers", size="sm")
        btn3 = gr.Button("Zen dining in Little Tokyo?", size="sm")

    msg_input.submit(handle_chat, [msg_input, chatbot], [chatbot])
    msg_input.submit(lambda: "", None, msg_input)

    btn1.click(handle_chat, [gr.State("Find me some moody restaurants"), chat
    btn2.click(handle_chat, [gr.State("Tell me about Iron & Embers"), chatbot
    btn3.click(handle_chat, [gr.State("What's a zen dining experience in Litt
```

The three quick-start buttons use `gr.State` to inject a pre-written message directly into `handle_chat` without requiring the user to type anything. `msg_input.submit` clears the text box after sending by binding a second handler that returns an empty string.

Entry point

Add the entry point that launches the app:



python

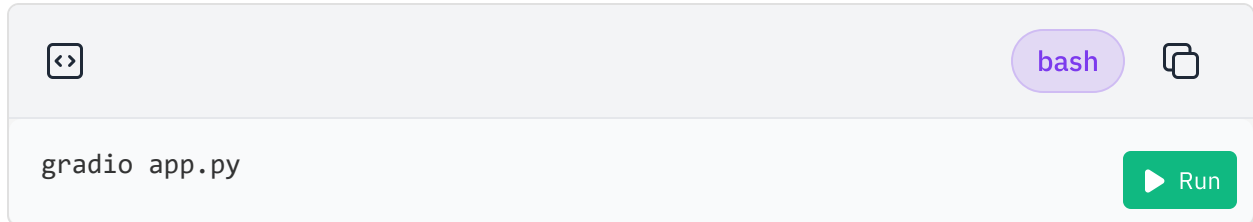


```
# Launch the App
if __name__ == "__main__":
    print("Starting Connoisseur Companion...")
    demo.launch(
        share=True,
        theme=gr.themes.Soft(),
    )
```

`share=True` generates a public Gradio URL so the app can be accessed from any browser. `gr.themes.Soft()` applies a clean, polished visual theme.

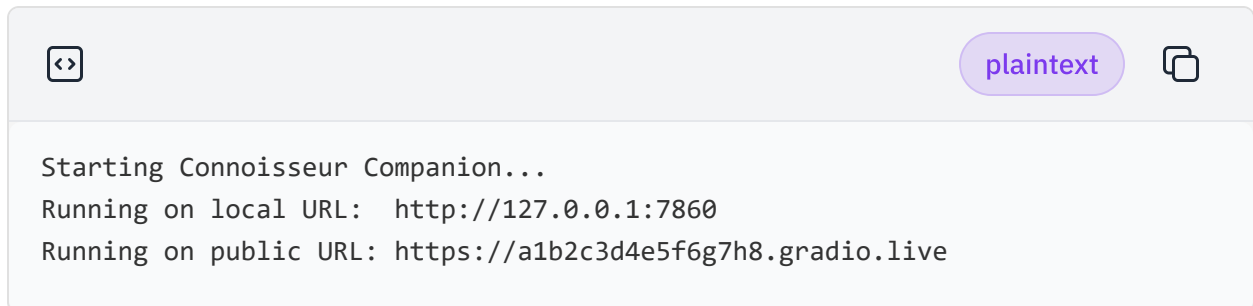
Run the application

Launch the app with Gradio's CLI:



A terminal window with a light gray header bar. On the left is a code icon. On the right is a purple pill-shaped button labeled 'bash' and a copy icon. The main area contains the text 'gradio app.py'. At the bottom right is a green button with a play icon and the text 'Run'.

Gradio will print output similar to the following in your terminal:



A terminal window with a light gray header bar. On the left is a code icon. On the right is a purple pill-shaped button labeled 'plaintext' and a copy icon. The main area contains the following text:
Starting Connoisseur Companion...
Running on local URL: http://127.0.0.1:7860
Running on public URL: https://a1b2c3d4e5f6g7h8.gradio.live

Open the **public URL** (the `gradio.live` link) in your browser. The exact URL will be different each time you launch the app. Once the interface loads, try asking a question or clicking one of the three quick-start buttons to see the agent reason through a tool call and return a response!

Take a screenshot of the entire Gradio user interface (UI) you created and name it `M4L3_Design_LLM_MCP_Host.jpg`. This screenshot will be used as one of the submission components later for your Final Project.

It should look something like this:

Connoisseur Companion

Your AI guide to California's restaurant scene. Ask me about restaurants by name, cuisine, or vibe!

Chatbot

Find me some moody restaurants

Find me some moody restaurants

Here are some moody restaurants that might pique your interest:

- Iron & Embers** in the Arts District, DTLA: This moody, industrial American steakhouse offers a 4.8/5 rating. The space features a 45-day dry-aged ribeye served with bone marrow chimichurri.
- Blue Bayou** in Long Beach: A 4.6/5-rated Cajun & Creole kitchen, Blue Bayou brings the flavors of New Orleans to the West Coast in a dim-lit room that often features live jazz on the weekends.
- Midnight in Milan** in Downtown LA: A sultry and fashionable Northern Italian restaurant, Midnight in Milan scores a 4.6/5 for its elevated take on risotto and veal osso buco. The moody, high-ceilinged room in the Fashion District attracts a stylish, late-night crowd.
- Taco Nocturna** in Whittier: A moody and artistic Mexican night-market stall turned permanent brick-and-mortar space, Taco Nocturna earns a 4.3/5 for its vampiro tacos and charcoal-grilled onions.
- The Velvet Raven** in Burbank: A moody and theatrical Gastropub with a slight gothic edge, The Velvet Raven scores a 3.9/5. It's beloved by the local studio crowd for its "Deathly Hallows" cocktails and truffle-dusted fries, offering a dim, candlelit escape from the bright California sun.

Each of these restaurants offers a unique atmosphere, making them perfect for a moody evening out.

Ask about restaurants

e.g., "Find me a moody spot in DTLA" or "Tell me about Sakura Garden"

Find moody restaurants

Tell me about Iron & Embers

Zen dining in Little Tokyo?

Use via API  · Built with Gradio  · Settings 

Conclusion

You have built a complete **MCP host application**—a Gradio chat interface backed by a WatsonX LLM that discovers and calls MCP tools through a ReAct agent loop.

By completing this lab, you have:

- Connected to an MCP server at runtime and **discovered its tools dynamically**
- Converted MCP tool schemas into **LLM-compatible tool definitions** and bound them to the model
- Implemented a **ReAct loop** that iterates tool calls until the LLM produces a final answer
- Built a **Gradio chat UI** with a streaming placeholder and quick-start prompt buttons

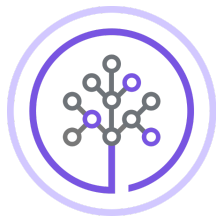
Together with the server built in the first lab, this application forms a complete end-to-end MCP system: a data layer exposed over the Model Context Protocol, and an intelligent host that uses it to answer natural language questions.

Author(s)

[Abdul Fatir | Data Scientist @ IBM](#)

©IBM Corporation. All rights reserved.

Lab: Build a Full MCP Application



Skills
Network

Estimated time: 30 minutes

Introduction

In this lab, you will build a complete MCP host application that integrates a Gradio chat interface with a WatsonX LLM. Using a ReAct agent loop, the system analyzes user questions, determines which MCP tools are needed, invokes them, and converts the results into clear natural language responses.

Screenshot requirement for the Final Project

During the course of the lab, you will be instructed to take a screenshot showing the full Gradio UI, including the application header, a visible chat history with at least one exchange, input/action buttons, and complete branding/footer. Name the file **M4L3_Design_LLM_MCP_Host.jpg**.

This screenshot will later serve as a component for the Final Project.

Learning objectives

By the end of this lab, you will have:

- Built an **MCP host** that discovers server tools at runtime and binds them to an LLM
- Implemented a **ReAct agent loop** that calls MCP tools iteratively until a final

- Wired the agent into a **Gradio chat UI** with quick-start prompt buttons
- Run the full application end-to-end from a single command

Setup →